Improving Vision Transformer's Time Perfomace via Linear Attention and Reinforcement Learning Soft-Gating

Chi-Van Vong, Minh-Quan Pham Nguyen
University of Science, Vietnam National University Ho Chi Minh City, Vietnam
{vcvan2424,pnmquan2414}@apcs.fitus.edu.vn

Abstract

While recently, Vision Transformers (ViTs) achieve state-of-the-art performance, their quadratic self-attention mechanism causes immense computational costs. Recent linear attention mechanisms mitigate the quadratic bottleneck but still redundantly process uninformative background tokens, wasting computational budget and limiting the model accuracy. In this paper, we propose a Multi-Agent Reinforcement Learning (MARL) gating framework to assign the importance of each token. Each visual token acts as an autonomous agent, utilizing a shared Actor-Critic policy to output continuous survival weights, gating information flow into the recurrent memory.

1. Introduction

In recent years, **Transformer** [15] has become a powerful foundational architecture for many models. **Vision Transformers** [2], one of the leading architectures in Computer Vision (CV), have made significant breakthroughs in object detection, image classification, and semantic segmentation. However, softmax attention - the core mechanism of Transformer, has a high computational complexity of $O(n^2)$. To address the issue, many works attempted to group vision tokens. Although this method reduces the complexity, Transformer loses its ability to perceive information globally. Other works replace softmax attention with **linear attention** which help reduce the quadratic complexity. However, linear attention inability to model complex information yields unsatisfactory results due to its low-rank nature and magnitude negligence. Many works have attempted to improve linear attention by tackling its low-rank nature [3]. Linear attention replaces softmax with kernal feature maps which causes the loss of sharpness. Therefore it can not be as selective as softmax attention which can place most of its attention weights on a few relevant tokens. To address this, we apply a Reinforcement Learning (RL) **gating** mechanism for each token, and extend to **token pruning** [9, 12] which help refine the input for linear attention. With this approach, we aim to speed up the training and inference time of the model without sacrificing its performance.

2. Related Work

2.1. Vision Transformer

Transformer [15] model is initially created for NLP tasks and achieved remarkable success. However, they did not have much impact on CV until Alexey Dosovitskiy *et al.* [12] introduced Vision Transformer (ViT), which splits the image into many small patches and feed as tokens to the model. Although this approach demonstrated strong performance in many CV tasks, its quadratic complexity due to self-attention making it computationally expensive to process high resolution images. To address this, Microsoft Research proposed Swin Transformer [8], which restricts attention to local windows, inspiring subsequent window-based variants. Other methods reduce attention cost by compressing keys or values using linear projections, strided convolutions, pooling, or patch clustering [1, 5–7, 10, 14]. Additionally, some variants explore alternative designs such as channel attention and softmax-free attention mechanisms.

2.2. Linear Attention

Linear Attention [14] reduces the quadratic complexity of softmax attention by using kernel functions to approximate $Sim()$ operator which reduce the complexity to linear. Given an input sequence of N tokens, standard self-attention computes

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V \tag{1}$$

where $Q, K, V \in \mathbb{R}^{N\times d}$ are the query, key, and value matrices and d is the feature dimension.

Linear attention replaces softmax operation with a kernel feature mapping $\phi(\cdot)$ applied element-wise

$$A = \phi(Q)\phi(K)^T V \tag{2}$$

Due to matrix rank constraints,

$$\text{rank}(A) \leq d. \tag{3}$$

when the number of tokens N is significantly larger than d, the attention mechanism must compress the interactions among many tokens into a limited representation. This compression can reduce the model's ability to capture fine-grained relationships among tokens, especially in high-resolution vision tasks. Thus its performance is still behind softmax. FLattenTransformer [5] introduces a focused function to enhance the ability of linear attention to concentrate on relevant information, while MLLA [6] establishes a link between linear attention and the Mamba model [4]. Many other approaches either rely on kernel-based approximations of the softmax function or integrate depthwise convolution to improve feature diversity. Recently, Qihang Fan *et al*. [3] addresses a fundamental limitation of linear attention by identifying the low-rank nature of its feature representations as a major cause of performance degradation. To overcome this, the paper proposes Rank-Augmented Linear Attention (RALA), which enriches both the key-value buffer and output features to raise their effective rank.

2.3. Token Pruning

ViT suffers from high computational cost propotional to the number of input tokens, hence many attempts have been made to reduce the redundant tokens while keeping the high performance. Traditional dynamic methods like DynamicViT [12] and TokenLearner [13] adaptively drop or condense tokens. Adaptive pruning technique such as AdaViT [11] use attention metrics or learned masks to select important tokens. Recent methods introduce the use of RL to learn the pruning policies. Specifically, Chenglong Lu *et al*. [9] formulates token pruning as a sequential decision making problem with multi-agent RL, where each of the visual token is consider an agent.

3. Method

We adopt the method from RALA[3] for solving the low rank problem, and replace the gating component with a multi-agent RL gating framework with every token treated as an agent sharing a continuous, stochastic Actor-Critic module. We aim to add a budget constraint, and layer wise information into the gating to improve the linear attention speed and performance. The gating score can be used to prune the token when inferencing.

3.1. Architecture Overview

An input image is tokenized into N patch embeddings and partitioned into T sequential chunks of size C. A linear-attention ViT backbone processes each chunk, projects it into local queries Q_t, keys K_t, and values V_t and maintain a recurrent memory (S_t, Z_t) in place of a full attention matrix. The shared Actor-Critic module evaluates the local token features in parallel. At each layer l, which represents a discrete time step in the Markov Decision Process (MDP), every agent $n \in \{1, \ldots, N\}$ produces a scalar gating weight $w_n \in (0, 1)$. For a specific chunk t, the agents collectively produce a weight vector:

$$\mathbf{w}_t = [w_{t,1}, w_{t,2}, \ldots, w_{t,C}]^\top \in (0,1)^C \tag{4}$$

To perform the row-wise memory update, we compute a chunk-level importance factor $\bar{w}_t$ by averaging the actions of the C agents within that specific chunk:

$$\bar{w}_t = \frac{1}{C} \sum_{c=1}^{C} w_{t,c} \tag{5}$$

These weights are then utilized to gate both the parallel memory update and the recurrent decay:

$$S_t = (1 - \gamma(1 - \bar{w}_t))S_{t-1} + (\mathbf{w}_t \odot \phi(K_t))^\top V_t \tag{6}$$

After all T chunks are processed via the sequential recurrence, the final state (S_T, Z_T) is consumed by the task head to produce the terminal reward R, which is used to guide the cooperative policy optimization.

Training follows a two-phase curriculum. **Phase 1** pretrain the ViT on the classification task to obtain a visual backbone. Then the RL agents is supervised trained with the Fisher Information of the pretrained ViT, avoiding the policy collapse that arises when RL is applied cold to a high-dimensional visual state. **Phase 2** fine-tunes the full system end-to-end on the downstream task via Proximal Policy Optimization (PPO) with Generalized Advantage Estimation. During inference, stochastic sampling is disabled, and the Actor produces deterministic continuous weights with hard pruning per chunk.

3.2. Chunkwise Formulation of Linear Attention

Standard linear attention leverages the associative property to reduce complexity to $\mathcal{O}(N)$. The output for the i-th query is typically computed using a global normalization term:

$$O_i = \frac{\phi(Q_i) \sum_{j=1}^{N} \phi(K_j)^\top V_j}{\phi(Q_i) \sum_{j=1}^{N} \phi(K_j)^\top} \tag{7}$$

To enable sequential importance assignment, we reformulate this into a causal, chunkwise process. The input sequence of N tokens is partitioned into T non-overlapping sequential chunks of size C. At each step $t \in \{1, \ldots, T\}$, the backbone processes the current chunk into local Queries (Q_t), Keys (K_t), and Values (V_t).

Rather than computing a full sequence matrix, we maintain two recurrent memory representations to capture global context. The numerator state $S_t \in \mathbb{R}^{d \times d}$ accumulates the

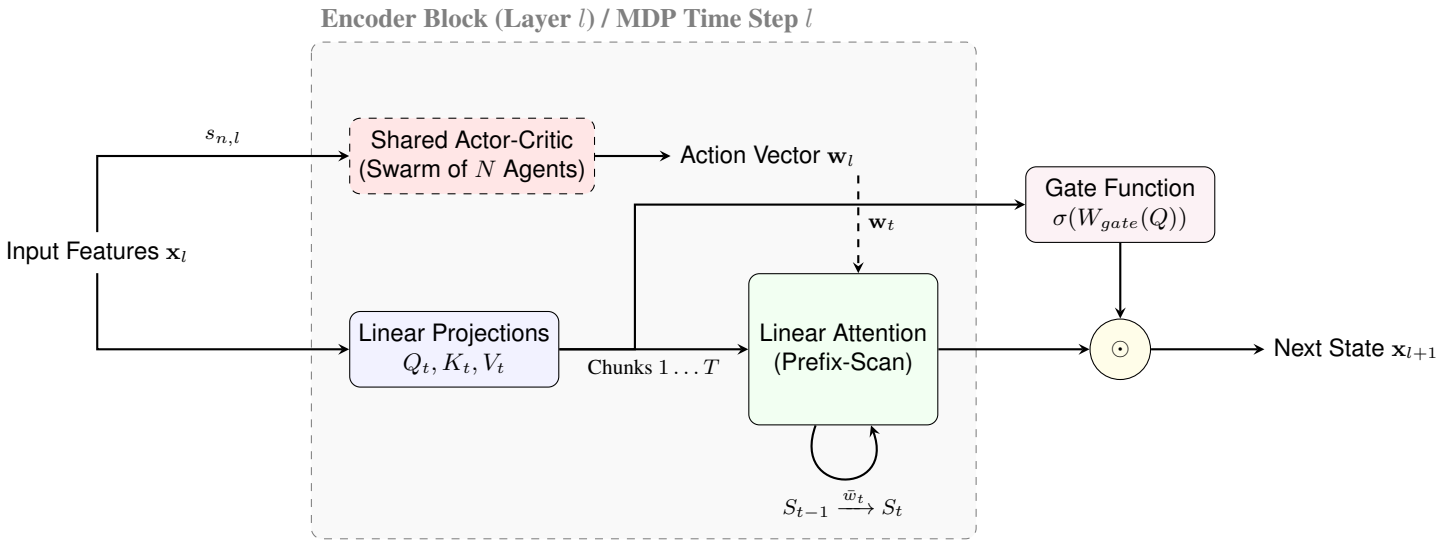

Figure 1. Architecture of the MARL-Gated Linear Attention Block. At each layer l, the swarm of N agents independently evaluates the local features to produce survival weights $\mathbf{w}_l$. These weights dictate the information flow into the chunkwise recurrent memory S_t. Finally, the intermediate linear attention output is refined via a Hadamard-gated transformation to produce the state x_{l+1} for the next layer.

cross-covariance of keys and values, while the denominator state $Z_t \in \mathbb{R}^{d\times 1}$ tracks the normalization factor:

$$S_t = S_{t-1} + \phi(K_t)^\top V_t \tag{8}$$

$$Z_t = Z_{t-1} + \sum_{i\in \text{chunk } t} \phi(K_{i,t})^\top \tag{9}$$

Following RALA [3], we use the Hadamard transformation to fix the low rank problem of the linear attention:

$$\phi(X) = \sigma(HX) \tag{10}$$

where H denotes a Hadamard matrix and $\sigma(\cdot)$ is a positive activation function.

The attention output for the current chunk t is then computed as:

$$O_t = \frac{\phi(Q_t)S_t}{\phi(Q_t)Z_t} \tag{11}$$

3.3. Markov Decision Process Formulation

By reformulating linear attention into a blockwise execution, we isolate the importance evaluation into a continuous MDP. To avoid the autoregressive bottleneck comed with sequential RL evaluation, we isolate the gating policy from the recurrent state update, enabling parallel associative scans across the sequence.

3.3.1. State Space

To enable parallel execution across all T chunks, the agent's observation is localized. For each layer l (representing the temporal step), the state $s_{n,l}$ for agent $n \in \{1, \dots, N\}$ is derived directly from the local visual features:

$$s_{n,l} = \text{Proj}(x_{n,l}), \tag{12}$$

where $x_{n,l}$ is the feature embedding of the n-th token at layer l, and $\text{Proj}(\cdot)$ is a shared lightweight projection.

3.3.2. Action Space

The swarm of N agents utilizes a shared Actor-Critic policy π_θ to evaluate the tokens in parallel. Each agent n outputs a scalar continuous weight $w_{n,l} \in (0, 1)$, representing its relative importance to the task. For a specific chunk t, the agents produce a collective action vector $\mathbf{w}_t$:

$$\mathbf{w}_t = [w_{t,1}, w_{t,2}, \dots, w_{t,C}]^\top \tag{13}$$

3.3.3. Reward Function

The swarm is governed by a global cooperative reward function calculated at the terminal layer L. To enforce the computational budget B_{target} without hard pruning, we define the total expenditure as the empirical mean of the weights:

$$B_{\text{used}} = \frac{1}{L \cdot N} \sum_{l=1}^{L} \sum_{n=1}^{N} w_{n,l} \tag{14}$$

The terminal reward R:

$$R = R_{task} - \lambda \cdot \max(0, B_{\text{used}} - B_{\text{target}}), \tag{15}$$

where λ is a penalty coefficient. This penalizes the swarm at the end of the forward pass if their collective survival weights exceed the target capacity, forcing agents to aggressively dropping the weight while maintaning the model performance

3.3.4. Squashed Gaussian Actor-Critic Network

To enable continuous down-weighting of uninformative features, we model the swarm's policy $\pi_\theta(a_{n,l} \mid s_{n,l})$ as a **Squashed Gaussian** distribution. This approach avoids the non differentiable of discrete gatings and the competitive constraints of global Softmax operations, allowing each agent to determine its survival weight independently.

The shared Actor network computes the parameters of an unbounded Normal distribution for every agent $n \in \{1, \ldots, N\}$ at layer l. To ensure numerical stability, the standard deviation is passed through a Softplus activation with a small ϵ floor:

$$\mu_{n,l} = \mathrm{MLP}_\mu(s_{n,l}), \quad \sigma_{n,l} = \mathrm{Softplus}(\mathrm{MLP}_\sigma(s_{n,l})) + \epsilon \tag{16}$$

During training, actions are sampled using the reparameterization trick to allow backpropagation through stochastic nodes. The raw sample $x_{n,l}$ is squashed via a hyperbolic tangent and shifted into the valid continuous weighting space of $(0, 1)$:

$$w_{n,l} = \frac{\tanh(x_{n,l}) + 1}{2}, \quad x_{n,l} \sim \mathcal{N}(\mu_{n,l}, \sigma^2_{n,l}) \tag{17}$$

By decoupling the weights from a global sum, the swarm can collectively silence redundant spatial regions (where $w_{n,l} \to 0$) or amplify high-utility features. However, the non-linear Tanh transformation alters the probability density. To maintain the mathematical integrity of the PPO objective, we apply the **Jacobian correction** for the log-probability:

$$\log \pi(w_{n,l}) = \log p(x_{n,l}) - \log \left(\frac{1 - \tanh^2(x_{n,l})}{2} + \epsilon \right) \tag{18}$$

Simultaneously, a shared Value network (Critic) $V_\phi(s_{n,l})$ estimates the expected episodic return. This provides a baseline to calculate the advantage $\hat{A}$, reducing variance during the cooperative policy update.

3.4. Pretraining: Information-Saliency Distillation

Initializing a swarm of RL agents from scratch in a high-dimensional visual space often leads to representation collapse, as random initial actions can destroy the backbone's recurrent state. To circumvent this, Phase 1 initializes the shared Actor-Critic module using empirical **Fisher Information** ($\mathcal{I}_n$) as a saliency oracle.

After warming up the backbone on a dense classification task, we compute the squared gradients of the cross-entropy loss with respect to the token features to approximate their informational utility. These scores, normalized to $\tilde{\mathcal{I}}_n \in [0, 1]$, represent the survival value for each agent. The Actor and Critic are then pretrained via supervised learning to mimic this oracle. The Actor minimizes the Mean Squared Error (MSE) between its deterministic mean prediction and the Fisher utility, while the Critic learns the expected saliency of the state:

$$\mathcal{L}_{\text{Actor}} = \mathbb{E}\left[\sum_{n=1}^{N} \left(\mu_{n,l}(s_{n,l}) - \tilde{\mathcal{I}}_n \right)^2 \right] \tag{19}$$

$$\mathcal{L}_{\text{Critic}} = \mathbb{E}\left[\sum_{n=1}^{N} \left(V_\phi(s_{n,l}) - \sum_{j=1}^{N} \tilde{\mathcal{I}}_j \right)^2 \right] \tag{20}$$

where $s_{n,l}$ is the local feature observation of agent n at layer l, and $\tilde{\mathcal{I}}_j$ is the ground-truth Fisher saliency for token j. By forcing every agent's local Value head to predict the global saliency sum, the swarm develops a shared understanding of the total information available in the image.

3.5. Finetuning: Cooperative Policy Optimization

In Phase 2, the full system is optimized end-to-end via PPO. Aggressive down-weighting of tokens by the swarm can cause the recurrent memory S_t to stagnate. To mitigate this, we augment the memory update with a **state dilution term** Γ_τ, which represents a fraction of the historical average scaled by the degree of suppression $(1 - \bar{w}_\tau)$:

$$\Gamma_\tau = \gamma \cdot (1 - \bar{w}_\tau) \cdot \frac{S_{\tau-1}}{\tau - 1} \tag{21}$$

When the swarm collectively suppresses a chunk ($\bar{w}_\tau \to 0$), Γ_τ tops up the memory with the mean historical contribution, preventing gradient vanishing.

Using Generalized Advantage Estimation ($\hat{A}$) derived from the Critic, the swarm is updated via the clipped PPO objective. To prevent premature policy convergence, we inject a Jacobian-corrected entropy bonus for the squashed Gaussian distribution:

$$\mathcal{L}_{\text{PPO}} = -\mathbb{E}\left[\min\left(r(\theta)\hat{A},\ \mathrm{clip}(r(\theta), 1 \pm \epsilon)\hat{A} \right) \right] - \lambda_{\text{ent}} H(\pi_\theta), \tag{22}$$

where the entropy $H(\pi_\theta)$ accounts for the Tanh-squashing transformation:

$$H(\pi_\theta) = \frac{1}{2}\log(2\pi e \sigma^2) + \mathbb{E}\left[\log\left(\frac{1 - \tanh^2(x)}{2} + \epsilon \right) \right] \tag{23}$$

During inference, stochastic sampling is disabled. The shared Actor deterministically produces the weight vector $\mathbf{w}_l \in (0, 1)^N$ for each layer.

4. Experiments

4.1. Experimental Design

Our evaluation includes two primary CV tasks: **image classification** and **object detection**. We focus on measuring

the trade-off between model accuracy and computational efficiency (both training and inference) while conducting ablation studies to isolate the impact of each architectural component.

4.2. Task Specifications

Image Classification We use the **ImageNet-100** data a standard benchmark containing over 100 thousand ages across 200 categories. Performance is measured u **Top-1 Accuracy**, which denotes the percentage of instar where the model's highest-probability prediction mat the ground-truth label.

Object Detection For localization tasks, we use **CO** 2017 which is widely recognized for its complexity and riety of object scales. We report **Average Precision (** a metric that accounts for both the precision and recal the predicted bounding boxes at various Intersection Union (IoU) thresholds.

4.3. Implementation Environment

The framework is implemented in **Python** using **PyTorch**. All experiments, including training and inference profiling, are conducted on a local computer with a single NVIDIA RTX 5060Ti 16GB.

4.4. Results

- object detection baselines: RALA, Ours, RMT-T.
- image classification baselines: RALA, Ours, MLLA-T.
- inference speed

4.4.1. Image Classification

Cost	Model	Params(M)	FLOPs(G)	Top1-acc(%)
$\sim 2.5G$	RALA			
	RMT-T			
	Ours	5.67	1.71	**48.62**

Table 1. Classification results of different general backbones on ImageNet-100

4.4.2. Object Detection

Cost	Model	Params(M)	FLOPs(G)	Top1-acc(%)
$\sim 2.5G$	RALA			
	MLLA-T			
	Ours			

Table 2. Object detection results of different general backbones on COCO 2017

4.4.3. Inference Speed

The inference speed of our model is in in Figure 2

Figure 2. The speed and GFLOPS with respect to image size

4.5. Ablation Study

4.5.1. Other gating mechanism

Discuss the result of other gating mechanism compare to RL. We hypothesize that the RL gating will be superior to the existing methods due to its ability to gather global information. On the other hand, the existing methods may have better speed, but it should be negligible. We can see that

Cost	Model	Params(M)	FLOPs(G)	Top1-acc(%)
$\sim 2.5G$	No Gating	5.67	1.71	**46.30**
	RALA			
	Ours	5.67	1.71	**48.62**

Table 3. Classification result of different gating mechanisms

the MARL gating help remove the noise tokens within the linear attention memory, hence improving the accuracy.

4.5.2. Without budget concern

Discuss the result fo RL gating without budget concern. We hypothesize that the RL will be lazy to assign a low weight for tokens, since it only needs to retain the performance.

4.5.3. Without Fisher Initialization

Discuss the training time, result without Fisher Initialzatio for Actor-Critic network. We propose that training the Actor-Critic network will be difficult due to unstablized training, and the high variance of RL.

4.6. Qualitative Study

4.6.1. Gating Map Visualization

To qualitatively evaluate the token selection behavior of our MARL framework, we visualize the spatial gating maps generated by the swarm's continuous actions. Figure 3,4 illustrates the original input images alongside their corresponding survival weight heatmaps ($w_{n,l}$) at the deeper layers of the network. We can observe that the agents successfully learn to distinguish task-salient features from redundant background noise. High survival weight are densely concentrated on the primary subjects. Conversely, the background regions are assigned near-zero weights, silencing them from the recurrent memory update.

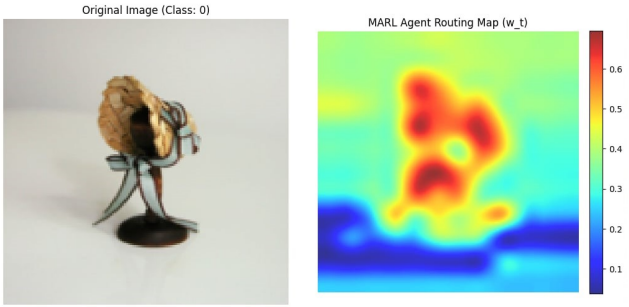

Figure 3

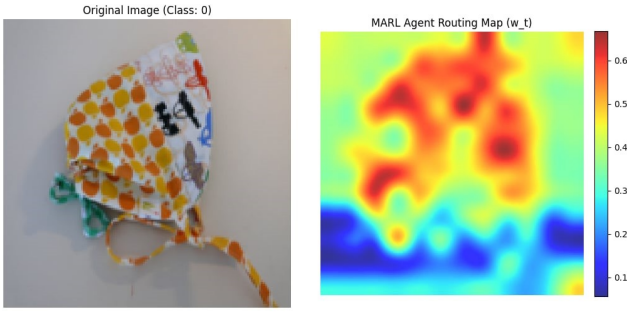

Figure 4

5. Threats To Validity

In this study, several threats to validity must be acknowledged.

A primary assumption of our Phase 1 pretraining is that empirical Fisher Information serves as an accurate proxy for ground-truth visual saliency. Gradients from a cross-entropy loss may not fully capture complex, multi-token spatial dependencies. Furthermore, token pruning can result in a worse accuracy, especially with the structure of chunkwise linear attention.

The inherent variance of Proximal Policy Optimization poses an internal threat. RL algorithms are notoriously sensitive to hyperparameter configurations, specifically the sparsity penalty coefficient (λ) and the entropy bonus.

Current evaluation is primarily constrained to image classification tasks. It remains to be empirically validated how well this localized, independent agent formulation generalizes to dense prediction tasks, such as high-resolution object detection or semantic segmentation, where maintaining precise background context is often more critical than in image-level classification.

6. Conclusion

We proposed a MARL gating framework to add a budget constraint and gather global information in order to improve the Linear Attention ViT models. In the current state, the framework is not fully developed yet, but still show a good heat map and increase the accuracy of the baseline model. In the future, we will refine the framework and try to experiment on more baseline models.

References

[1] Han Cai, Junyan Li, Muyan Hu, Chuang Gan, and Song Han. Efficientvit: Multi-scale linear attention for high-resolution dense prediction. In *ICCV*, 2023. 1

[2] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2020. 1

[3] Qihang Fan, Huaibo Huang, and Ran He. Breaking the low-rank dilemma of linear attention. In *CVPR*, 2025. 1, 2, 3

[4] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2024. 2

[5] Dongchen Han, Xuran Pan, Yizeng Han, Shiji Song, and Gao Huang. Flatten transformer: Vision transformer using focused linear attention. In *ICCV*, 2023. 1, 2

[6] Dongchen Han, Ziyi Wang, Zhuofan Xia, Yizeng Han, Yifan Pu, Chunjiang Ge, Jun Song, Shiji Song, Bo Zheng, and Gao Huang. Demystify mamba in vision: A linear attention perspective. In *NeurIPS*, 2024. 2

[7] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *ICML*, 2020. 1

[8] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *ICCV*, 2021. 1

[9] Chenglong Lu, Shen Liang, Xuewei Wang, and Wei Wang. Reinforcement learning-based token pruning in vision transformers: A markov game approach. In *ICME*, 2025. 1, 2

[10] Jiachen Lu, Junge Zhang, Xiatian Zhu, Jianfeng Feng, Tao Xiang, and Li Zhang. Softmax-free linear transformers. In *IJCV*, 2024. 1

[11] Lingchen Meng, Hengduo Li, Bor-Chun Chen, Shiyi Lan, Zuxuan Wu, Yu-Gang Jiang, and Ser-Nam Lim. Adavit:

Adaptive vision transformers for efficient image recognition. In *CVPR*, 2022. 2

[12] Yongming Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. Dynamicvit: Efficient vision transformers with dynamic token sparsification. In *NeurIPS*, 2021. 1, 2

[13] Michael S Ryoo, AJ Piergiovanni, Anurag Arnab, Mostafa Dehghani, and Anelia Angelova. Tokenlearner: What can 8 learned tokens do for images and videos? *arXiv preprint arXiv:2106.11297*, 2021. 2

[14] Zhuoran Shen, Mingyuan Zhang, Haiyu Zhao, Shuai Yi, and Hongsheng Li. Efficient attention: Attention with linear complexities. In *WACV*, 2021. 1

[15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017. 1